GymTracker

Architecture description

Code-grounded description of the solution as it exists. Analysis only. No design was changed to produce this document.

Solution: `GymTracker.sln`
Runtime: .NET 8, Blazor Interactive Server, EF Core 8.0.11, SQL Server, MudBlazor 9.1.0, ASP.NET Core Identity
Date: 5 October 2026

How to read this

Each major section uses the same order: what the part is for, a glance at the pieces, the implementation, then a walk through a real path in the application.

1. Overall system architecture
2. UI / presentation architecture
3. Business service architecture
4. Data access / generic repository architecture
5. Data model architecture
6. Template, schedule, and actual architecture
7. Personalized schedule feature
8. Workout execution and logging
9. Database and EF Core architecture
0. Transaction and consistency model

1. Overall system architecture

1. High-level overview

GymTracker is a personal gym application. A signed-in user keeps a catalogue of exercises and workout templates, keeps a weekly plan of which template to do on which day and period, starts today's workout from that plan, and records what was actually performed. A newer path lets the user state a fitness level and training goals, receive a draft weekly plan, edit that draft, and only then replace the live weekly plan.

The application is a single web host. Pages are Blazor components rendered on the server. Those pages call C# services. Services either orchestrate several data operations or map between screen models and database rows. Persistence goes through Entity Framework Core into SQL Server. User accounts are a separate Identity database context that shares the same connection string and does not own the workout tables.

Two persistence styles live side by side. Scheduling features receive one shared database context for the web request and stage several changes before one save. Older workout, exercise, schedule, and logging services open a new database context inside each method and save on every insert, update, or delete.

Read this section as the map. Later sections explain one layer at a time. The UI never talks to SQL. Business services do not render pages. The generic repository does not know about proposals, logging, or templates. The data model describes the stored shape. Identity answers "who is logged in"; workout rows store that person's identifier as a Guid with no foreign key back to the account table.

2. Architecture at a glance

```
Browser
  ↓
Blazor Interactive Server pages (MudBlazor)
  ↓
Business services and view models
  ↓
IGenericRepoService<T>   or   a new EntityDBContext per older method
  ↓
GenericRepo<T>
  ↓
EntityDBContext          ApplicationDbContext (Identity only)
  ↓                        ↓
SQL Server, one connection string (DefaultConnection)
```

Major responsibilities:

Layer	Responsibility
Web host	Composition, authentication cookies, routing, MudBlazor, both database contexts
Pages	What the user sees and which service method runs after a click
Business services	Workflows, validation, mapping, and the decision of when to write
Generic data access	Query, stage, save, and transaction against one entity type
Data model	Entity classes, keys, relationships, and delete behaviour
Migrator	One-off seeding of lookup data, templates, and a novice week
Tests	xUnit checks of services, often on EF Core's in-memory provider

3. Detailed breakdown

The solution file `GymTracker.sln` contains eight projects. A folder named `MudBlazor.Examples.Data` exists on disk, targets .NET 10, and is not a project in the solution.

Project	Type	Responsibility	Depends on	Depended on by	Notable packages
GymTracker	ASP.NET Core web, Blazor	Host, pages, Identity, DI registration	BusinessService, DataModel, DbUow, Helpers, Client	The running site	MudBlazor 9.1.0, Identity EF 8.0.3, EF Core SqlServer 8.0.11
GymTracker.Client	Blazor WebAssembly	Small client assembly referenced for extra	MudBlazor, WebAssembly auth packages	GymTracker	Microsoft.AspNetCore.Components.WebAssembly 8.0.3

		routes (`Auth`, `Counter`, `RedirectToLogin`). The host renders interactive server components.			
GymTrackerBusinessService	Class library	Workflows, view models, older feature services	DataModel, DbUow	Web, Helpers, Tests	Components.Authorization 8.0.3
GymTrackerDbUow	Class library	`GenericRepo<T>`. There is no unit-of-work class.	DataModel	BusinessService, Web	EF Core via DataModel
GymTrackerDataModel	Class library	Entities, `EntityDBContext`, migrations, UI attributes	None of the other projects	Everyone that touches data	EF Core 8.0.11, SqlServer, Tools
GymTrackerHelpers	Class library	`ReflectionHelper` used by the grids	BusinessService, DataModel	Web, Tests, Migrator	FluentValidation 12.1.1
GymTrackerDataMigrator	Console exe	Seeds templates, goals, and the novice week	DataModel, Helpers	Run by hand	—
TestServices	xUnit exe	Service tests	BusinessService, Helpers	—	xUnit 2.9.2, EF Core InMemory 8.0.11

`Program.cs` in the web project registers `AddInteractiveServerComponents`, cookie authentication, `ApplicationDbContext` and `EntityDBContext` against `DefaultConnection`, Identity on `ApplicationUser` with confirmed accounts required, scoped `IGenericRepoService<>`, the workout services, the scheduling services, `CatalogScheduleAdvisor` as `IScheduleAdvisor`, MudBlazor, and `SmtpEmailSender`. Login and logout are minimal API posts at `/api/login` and `/api/logout`.

Namespaces are not a clean layer diagram. `GenericRepo<T>` lives in namespace `GymTrackerDbUOW` and implements `IGenericRepo<T>` in namespace `GymTrackerDbUow.Generic`. Feature services live in `GymTrackerBusinessService.Repository` even when they are workflows. Scheduling types live in `GymTrackerBusinessService.Scheduling`.

4. How it works in practice

```
User opens Today's Workout
  ↓
Blazor route /manage/logworkout in Manage.razor
  ↓
LogWorkout asks ILogWorkOutService which DOWWorkout matches now
  ↓
LogWorkOutService opens its own EntityDBContext and reads TRN_DOWWorkout
  ↓
User completes a set
  ↓
CompleteSet copies exercise name and planned metric values into Actual* rows
  ↓
SQL Server
```

A second path, Generate Schedule, stays on the scoped context registered in `Program.cs`. That difference matters: two services in one request share one `EntityDBContext` only when both were constructed by DI. A logging method that calls `new EntityDBContext(Helpers.BuildOptions())` has its own context and its own save.

2. UI / presentation architecture

1. High-level overview

The UI is Blazor Interactive Server with MudBlazor. Most working screens are not hand-built forms. A page chooses an entity or a view model, and shared components draw columns and editors by reflecting over public properties and custom attributes.

There are two kinds of screen. Catalogue and schedule maintenance go through `Manage.razor` into `ManageGrid`. That grid lists rows and opens `UpsertGridData` as a dialog. Generate Schedule is a custom page. It reuses the same dialog and the same property editor, but it does not use `ManageGrid`, and the dialog's Submit button does not write the database. The page buttons do.

View models exist where the screen shape is not the entity shape. A workout template is edited as a parent plus exercises plus sets. A proposal is edited as a header plus slot rows. History is a display model. Logging is a planned-versus-actual model. Simple lookups and the live weekly plan are edited closer to the entity itself.

From the user's point of view, persistence happens when a dialog submits on the Manage screens, and when Save changes or Accept is pressed on Generate Schedule. Edits inside a dialog stay in memory until that point. Cancel throws the dialog's copy away.

2. Architecture at a glance

```
MyNavMenu
  ↓
Manage.razor  (/manage/{EntityName})          GenerateSchedule.razor (/schedule/generate)
  ↓                                             ↓
ManageGrid<TEntity>                           page state (view models)
  ↓                                             ↓
UpsertGridData dialog                         same UpsertGridData dialog
  ↓                                             ↓
GridPropertyEditor / ChildGridPropertyEditor  in-memory until Save changes or Accept
  ↓
service callback, then SQL
```

Signed-in links in `MyNavMenu.razor`: Home, Manage Workouts, Manage Exercises, Generate Schedule, Workout Schedule, Workout Records, Today's Workout. The older `NavMenu.razor` is not that menu. `Routes.razor` uses `AuthorizeRouteView` and `MainLayout`. The Generate Schedule page reads the user id from `AuthenticationStateProvider` and does not carry an `[Authorize]` attribute of its own; the menu link is inside `AuthorizeView`.

3. Detailed breakdown

Route dispatcher

`Manage.razor` is `@page "/manage/{EntityName}"`. It branches on the route string:

Route	TEntity	Child models	Who loads and saves
exercise	Exercise	EmptyGridModel	ManageGrid loads via generic repo. Header save calls ExerciseService.SaveExerciseData. DisplayAddEdit is false.
templateworkout	TemplateWorkout	WorkoutGroupVM, WorkoutSetVM	WorkoutExerciseSetService loads children and, on submit, writes header, exercises, and sets.
workoutschedule	DOWWorkout	EmptyGridModel	WorkoutScheduleService loads the user's rows and saves one row. DisplayAddEdit is false.
viewworkouthistory	WorkoputHeaderDisplayVM	EmptyGridModel	LoadData only. CanEditAddButton is false.
logworkout	—	LogWorkoutLogExerciseVM, LogWorkoutSetVM	LogWorkout component, not ManageGrid.

ManageGrid

`ManageGrid` injects `IGenericRepoService<TEntity>`. On first render it uses `LoadData` when the page supplied one, otherwise `repo.GetAllAsync()`. Columns come from properties that are primitive, string, `DateTime`, or a value type, and are not marked `IgnoreInGrid`, plus any property marked `NavigationProperty`. Operator precedence makes the navigation attribute win even when the property is a class. A display name of `Nr` is shown as the row index, not the stored id. `RenderAs(Link)` and `RenderAs(Video)` become links. Delete calls `repo.DeleteAsync` after `ConfirmDelete`, then reloads with `GetAllAsync` even when `LoadData` was used. Add and Edit open `UpsertGridData`.

ManageGrid is the right tool when the row type is an entity the generic repository can `Set<T>()`, and when add, edit, and delete mean "write that row now". It is the wrong tool for proposal slots. A slot list is a child collection of one draft, edits must remain unpersisted

until the page says so, and accepting the draft is a different write from editing a `DOWWorkout` row. Pointing ManageGrid at `ScheduleProposalSlot` would also treat slots as their own catalogue.

UpsertGridData and the property editors

`UpsertGridData` is a `MudDialog`. It cannot sit inline on a page; the page calls `DialogService.ShowAsync`. The header is reflected the same way as the grid. `ReadOnlyAttribute` makes string and int header fields read-only. Key fields stay read-only. `DayOfWeek` is a `MudSelect`. A navigation property becomes a `MudSelect` of `{Name}Id`, filled by `ReflectionHelper.GetNavigationData`, which loads the lookup table through `EntityDBContext` from DI. On first open, a navigation foreign key of 0 is set to the first lookup row. Child rows use `GridPropertyEditor`. Nested metric rows use `ChildGridPropertyEditor` and `ManageGridDetails`.

`GridPropertyEditor` handles navigation autocomplete, string, int, and `DayOfWeek`. The day select uses the current value when it is already a `DayOfWeek`, so Sunday (0) stays Sunday. A missing value falls back to Monday.

Submit runs `SaveHeaderData`, then `SaveDetailData` (a no-op if the page passed null), then `SaveGridDetailData`, then closes with `DialogResult.Ok`. On Manage Workouts those callbacks persist. On Generate Schedule they only copy the edited list back onto the page's clone. The dialog button text remains Submit on every screen. The Generate Schedule caption tells the user that Submit keeps edits on the page and Save changes stores the draft.

Attributes

Defined in `RenderAsAttribute.cs` in the data model, so entities and view models can share them:

- `RenderAsAttribute` — link or video.
- `IgnoreInGridAttribute` — hide the property. Foreign-key integers are usually hidden because the navigation is shown instead.
- `NavigationPropertyAttribute` — entity type, value field default `Id`, text field default `Name`. Display text in the grid uses Description, then Name. The TextField value is stored on the attribute and is not what the grid cell uses.
- `CollectionAttribute` — marks a nested detail collection, not a multi-select widget.
- `DetailPropertyAttribute` — a field that belongs in the child row.
- `SearchableAttribute` — search metadata.
- `ReadOnlyAttribute` from the framework — presence makes the editor read-only.

View models

Type	Role
WorkoutGroupVM	One exercise line on a template: exercise id, template-exercise id, and its sets.
WorkoutSetVM	One planned set: order and metric values.
WorkoutSetMetricVM	One planned metric on that set.
WorkoputHeaderDisplayVM	Read-only history row. The type name is spelled that way in code. Fields include workout name, day, start, end, status, notes.
LogWorkoutLogExerciseVM	One exercise while logging, loaded from the template.
LogWorkoutSetVM	One set, with planned text and a completion flag.
LogWorkoutSetMetricVM	Planned and actual metric values. Implements `IDynamicDisplayName` so the column title can follow the metric.
SchedulePreferenceVM / SchedulePreferenceGoalVM	Fitness level, read-only weekly-training text, and one row per goal.
ScheduleProposalVM / ScheduleProposalSlotVM	Draft header and editable sessions. No user id, status, or timestamps.
EmptyGridModel	Placeholder child type when a screen has no child grid.

4. How it works in practice

Editing a workout template:

```
User opens Manage Workouts
  ↓
ManageGrid lists TemplateWorkout rows from the generic repository
  ↓
Edit loads WorkoutGroupVM rows through GetHeaderDataAsync
  ↓
Expanding an exercise loads WorkoutSetVM rows through GetDetailDataAsync
  ↓
User adds, changes, or removes rows in the dialog
```

```
  ↓
Submit calls SaveWorkoutHeaderData, then SaveWorkoutExerciseData, then SaveExerciseSetsData
  ↓
The service compares the in-memory ids with the stored template-exercise ids
  ↓
Missing exercises are deleted; new ones are inserted; sets and metrics follow
  ↓
Each insert, update, or delete saves immediately on its own context
```

That is what the code does. It is a compare-and-write on submit, not a unit of work. `OnUpdateExerciseAsync` returns the current list unchanged; the real mapping happens in the save methods.

Editing a generated draft is the variation. The page clones `ScheduleProposalVM`, opens the same dialog, and on OK replaces the page state. Nothing is written until Save changes or Accept. Accept writes the slots first and then replaces `DOWWorkout`. Abandon does not save the in-memory list.

3. Business service architecture

1. High-level overview

The business-service project is the application's workflow layer. Some types are thin adapters that open a database context and insert or update one entity. Others coordinate several repositories, a validator, and an advisor, and they own the transaction. The project also holds the view models the screens edit, because those models are application data rather than database tables.

Scheduling services are injected and share the request's `EntityDBContext` through `GenericRepoService<T>`. They do not receive `EntityDBContext` in the scheduling classes themselves. Older services ignore that shared context. Each method builds options from configuration and constructs a new context. They filter in memory after loading a whole table when they need "this user's rows".

The advisor is a boundary inside this layer. `ScheduleProposalService` builds a request from preferences and the catalogue, `IScheduleAdvisor` returns a recommendation, and the validator decides whether that recommendation may be stored. The advisor does not write the database and does not see the user id, the live schedule, or workout history.

2. Architecture at a glance

```
Pages
  ↓
GenerateScheduleWorkflow ── TrainingPreferenceService
        │                    ScheduleProposalService
        │                         ├── ScheduleProposalValidator
        │                         ├── IScheduleAdvisor (CatalogScheduleAdvisor)
        │                         └── IGenericRepoService<T> for proposal, goals, slots, templates, periods, DOWWorkout
        │
Manage / LogWorkout
  ↓
WorkoutExerciseSetService, ExerciseService, WorkoutScheduleService,
WorkoutHistory, LogWorkOutService
  ↓
new GenericRepoService<T>(new EntityDBContext(...)) per method
```

3. Detailed breakdown

TrainingPreferenceService

Interface `ITrainingPreferenceService`. Methods `GetAsync(Guid userId)` and `SaveAsync(Guid userId, int fitnessLevelId, IReadOnlyCollection<int>? goalIds)`. Depends on generic services for `UserTrainingPreference`, `UserTrainingPreferenceGoal`, `FitnessLevel`, and `TrainingGoal`.

Get returns null when the user has no row. It does not create one. The DTO carries user id, fitness level id and name, min and max sessions, and goals ordered by id. It does not carry the weekly-training sentence. Save requires at least one goal, stores each goal id once, rejects unknown level or goal ids, and replaces the goal set on the existing row or inserts a new preference. The user id always comes from the method argument. The service stores the stated choices. It does not decide how many days to train. Save loads the fitness level as tracked and assigns both the foreign key and the navigation so EF does not insert a second level. The write is one `SaveAsync` inside `BeginTransactionAsync` / `CommitTransactionAsync` on the preference repository. UI consumer: `GenerateScheduleWorkflow`.

ScheduleProposalService

Interface `IScheduleProposalService`. Methods: `GenerateAsync`, `GetDraftAsync`, `UpdateSlotsAsync`, `GetWarningsAsync`, `AbandonAsync`, `AcceptAsync`. Depends on the preference service, generic services for the proposal, its goals, its slots, fitness level, template, template-goal, period, and `DOWWorkout`, plus `IScheduleAdvisor` and `ScheduleProposalValidator`.

`GenerateAsync` loads preferences, builds a catalogue of templates that match the user's level and share at least one selected goal, sends periods and that pool to the advisor, validates with `ValidateAiProposal` against every template id and period id, and only then replaces the user's draft. Empty or invalid advisor output stores nothing. `UpdateSlotsAsync` uses `ValidateUserProposal`, which does not require the slot count to equal the recommendation. It deletes the draft's slots and inserts the new list inside one transaction. `GetWarningsAsync` returns advisory text for session count outside the level range, a template from another level, a template whose goals miss the proposal, more than one slot on the same day and period, and an Evening period. Warnings do not block save or accept. `AbandonAsync` sets status Abandoned and `AbandonedAt` to `DateTime.Now`. `AcceptAsync` checks ownership and Draft status, validates the stored slots, removes only that user's `DOWWorkout` rows, adds rows from the stored slots, sets status Accepted and `AcceptedAt` to `DateTime.UtcNow`, and saves once. It does not read the screen's unsaved list. That is why the workflow saves first.

GenerateScheduleWorkflow

Interface `IGenerateScheduleWorkflow`. This is the page's façade. `LoadAsync` reads preferences, the draft, and warnings. `GenerateAsync` returns a confirmation result when a draft exists and the caller has not confirmed replacement; otherwise it saves preferences and generates. `SaveSlotsAsync` maps `ScheduleProposalSlotVM` to `ProposalSlotInput` and calls `UpdateSlotsAsync`. `AcceptAsync` calls `SaveSlotsAsync` and, only when that succeeds, `ScheduleProposalService.AcceptAsync`. `AbandonAsync` does not save slots. `PopulatePreferenceDisplayAsync` and `PopulateSlotDisplayAsync` fill detached display stubs so the summary can show names. UI consumer: `GenerateSchedule.razor`.

ScheduleProposalValidator

Not an interface. `ValidateAiProposal` checks slots and, when there is at least one slot, requires the recommended count to match. `ValidateUserProposal` checks slots only. Rules: at least one slot; template id exists; period id exists; day is a defined `DayOfWeek` (Sunday 0 through Saturday 6); the same proposal, day, period, and template may appear once. Two different templates on the same day and period are allowed. The validator does not look at session-count guidance.

IScheduleAdvisor and CatalogScheduleAdvisor

`RecommendAsync` takes `ScheduleAdvisorRequest` and returns `ScheduleAdvisorResponse`. The request has fitness level, session bounds, weekly-training text, goals, periods, and templates. It has no user id, age, sex, body measurements, current schedule, or history. The catalog implementation is deterministic and synchronous behind a completed task. Session count is `min + (max - min) / 2`, so Novice is 2, Intermediate is 3, and Advanced is 5. The period is Morning by name, otherwise the lowest id. Templates are ordered by same level and overlapping goals, then same level, then overlapping goals, then anything else; within a tier, more overlapping goals, then lower id. Days are spread: one Monday; two Monday and Thursday; three Monday, Wednesday, Friday; and so on up to all seven days with Sunday last. A template repeats only after the ordered list is exhausted. An empty catalogue or no periods returns count 0 and no slots, which the validator rejects. The explanation includes the weekly-training text when it is present. DI registers this class. There is no model-backed advisor in the solution.

WorkoutScheduleService

`LoadDOWDataPerUser` loads every `DOWWorkout` and filters to the user id in memory, ordered by day. `SaveWorkoutScheduleData` clears the period and template navigations, sets `UserId` from the argument, and inserts when `Id` is 0 or updates otherwise. Each call uses a new context. The Generate Schedule accept path does not call this service. The Workout Schedule page does.

WorkoutExerciseSetService and ExerciseService

`IWorkoutExerciseSetService` loads template exercises and sets into view models, and on submit diffs them back to `TemplateExercise`, `TemplateSet`, and `TemplateSetMetric`. Header save inserts or updates `TemplateWorkout`. `ExerciseService.SaveExerciseData` inserts or updates `Exercise`. Both use a new context per call. UI consumer: Manage pages.

LogWorkOutService and WorkoutHistory

`ILogWorkOutService` finds today's scheduled rows, starts or reuses an `ActualWorkout`, copies template exercise and metric values when a set is completed, and marks the workout ended. `WorkoutHistory.LoadWorkoutHistoryPerUser` projects `ActualWorkout` rows into `WorkoputHeaderDisplayVM`. Logging is described in section 8. These services construct their own contexts and, in logging, also keep table queries on a context field.

4. How it works in practice

```
User confirms Generate on /schedule/generate
  ↓
GenerateScheduleWorkflow.GenerateAsync saves preferences
  ↓
ScheduleProposalService.GenerateAsync builds the advisor request from the matching catalogue
  ↓
CatalogScheduleAdvisor.RecommendAsync returns days, one period, and template ids
  ↓
ScheduleProposalValidator.ValidateAiProposal
  ↓
ReplaceDraftAsync deletes the previous draft inside a transaction, then inserts the new draft
  ↓
Workflow reloads the draft into ScheduleProposalVM
  ↓
The page shows the draft. DOWWorkout is unchanged
```

4. Data access / generic repository architecture

1. High-level overview

Workout data is read and written through one generic stack. The stack exists so feature code can ask for "preferences", "slots", or "templates" without a repository class per table, and so several of those requests in one web request share one `EntityDBContext` and therefore one change tracker and one transaction.

```
Business service
  ↓
IGenericRepoService<T>     (GymTrackerBusinessService.Generic)
  ↓
GenericRepoService<T>      constructs GenericRepo<T> with the injected context
  ↓
IGenericRepo<T>            (GymTrackerDbUow.Generic)
  ↓
GenericRepo<T>             (namespace GymTrackerDbUOW)
  ↓
EntityDBContext
  ↓
EF Core
  ↓
SQL Server
```

Each layer is thin on purpose. `IGenericRepoService<T>` is what business services depend on, so they do not reference the repository project's concrete class. `GenericRepoService<T>` is the DI adapter: its constructor takes `EntityDBContext` and does `new GenericRepo<T>(dbContext)`. `GenericRepo<T>` is the only type that calls `Set<T>()`, `Include`, `SaveChanges`, and the database transaction. `EntityDBContext` is the EF model. The repository does not mention proposals, users, or workouts.

There is no `GymTrackerDbUow` class. "Uow" in the project name is historical. Immediate methods save before they return. Stage methods only attach work to the context. `SaveAsync` is `SaveChangesAsync` for everything staged on that context, including entities staged through a different `GenericRepo<T>` that was built with the same context instance.

2. Architecture at a glance

Operation	Writes immediately	Tracking
GetAllAsync	No	No. Includes properties marked NavigationProperty.
GetByIdAsync	No	Yes, via Find.
QueryAsync	No	Caller chooses. Include paths are strings, including nested paths such as Goals.TrainingGoal.
InsertAsync	Yes	Adds and saves. Returns the Id property, or 0 after a caught exception.
UpdateAsync	Yes	Nulls navigation properties, detaches a local copy, attaches the object as Modified, saves. A caught exception is written to debug output and not rethrown.
DeleteAsync	Yes	Find, Remove, SaveChanges. Exceptions are rethrown as Exception.
StageAdd / StageRemove	No	Add or Remove on the set.
SaveAsync	Yes, everything pending on the context	Returns the change count.
Begin / Commit / Rollback	Transaction only	On a relational database, uses Database.CurrentTransaction. If the provider is not relational, or a transaction is already open, Begin returns without starting one. Commit and Rollback do nothing when there is no current transaction.

3. Detailed breakdown

`GetAllAsync` reflects `NavigationPropertyAttribute` and calls `Include` on those property names only. It does not include collections that lack the attribute, and it does not include nested paths. That is why a schedule load can show period and template names, and why a proposal graph uses `QueryAsync` with explicit paths instead.

`QueryAsync` applies `AsNoTracking` when `trackChanges` is false, then each non-blank include path, then the predicate. Nested includes are ordinary EF strings: `"Slots.TemplateWorkout.FitnessLevel"`. Tracked queries are used when the service will edit the instances. Untracked queries are used for catalogues and for returning a draft to the UI.

`GetByIdAsync` uses `FindAsync`. A composite key can be passed as the object id because Find accepts a key value. `GetPrimaryKeyValue`, used by update, reads the EF key and returns one value or an array for a composite key.

`InsertAsync` saves immediately. On success it returns `Id` as an int. Entities whose key is not a property named Id, including `ScheduleProposalGoal`, are not inserted through this method. On failure it constructs an `InvalidOperationException`, does not throw it, and returns 0.

`UpdateAsync` sets every navigation-attributed property to null before attach. That is how a MudBlazor edit can update foreign keys without EF treating the navigation as a new lookup row. The catch block does not rethrow.

`StageAdd` and `StageRemove` are how scheduling replaces a draft, replaces slots, and replaces a user's `DOWWorkout` rows. Several repositories stage onto one context; one `SaveAsync` persists them together.

Tests use `UseInMemoryDatabase`. Named in-memory databases are shared by every context in the process that uses the same name. `Database.IsRelational()` is false, so transaction methods do nothing. A failure still prevents the store update when `SaveChanges` throws or an interceptor throws in `SavingChanges`.

DI in `Program.cs` is `AddScoped(typeof(IGenericRepoService<>), typeof(GenericRepoService<>))` beside `AddDbContext<EntityDBContext>`. One HTTP circuit scope therefore has one context. Older services opt out by constructing both the service and the context themselves through `Helpers.BuildOptions()`, which reads `DefaultConnection` from the configuration singleton.

4. How it works in practice

```
AcceptAsync on ScheduleProposalService
  ↓
QueryAsync tracked draft, including Slots
  ↓
QueryAsync tracked DOWWorkout rows for this user
  ↓
StageRemove each of those rows on the DOW repository
  ↓
StageAdd a new DOWWorkout per stored slot
  ↓
Set proposal status and AcceptedAt on the tracked draft
  ↓
_proposals.SaveAsync()
  ↓
One SaveChanges on the shared context
  ↓
CommitTransactionAsync
```

The DOW repository and the proposal repository are different generic services. They persist together because both were constructed with the same `EntityDBContext`.

5. Data model architecture

1. High-level overview

The stored world has four ideas. Configuration describes what a workout can contain and how hard it is aimed. Planning records what this user intends to do this week. A proposal is a draft of that plan, kept apart until the user accepts it. Actual rows record a session that was started, including names and planned numbers copied at the time, so later edits to a template do not rewrite history.

```
Configuration and templates
        ↓  referenced by id
User planning (preferences, proposal, live week)
        ↓  copied, not updated in place
Actual workout and its exercises, sets, and metrics
```

Identity users are not part of this model. `UserId` and `PerformedByUserId` are Guids. `ApplicationUser` has no gym profile fields.

2. Architecture at a glance

```
FitnessLevel ── TemplateWorkout ── TemplateExercise ── TemplateSet ── TemplateSetMetric
      │               │                                    │              │
      │               └── TemplateTrainingGoal ── TrainingGoal             └── Metric
      │
UserTrainingPreference ── UserTrainingPreferenceGoal ── TrainingGoal

ScheduleProposal ── ScheduleProposalGoal
        │
        └── ScheduleProposalSlot ── WorkoutPeriod
                    │
                    └── TemplateWorkout

DOWWorkout ── day, period, template, user   (live plan)

ActualWorkout ── ActualExercise ── ActualSet ── ActualSetMetric
      │               │              │
      └── optional template link     └── optional template-set link
                      └── Exercise (restricted)
```

3. Detailed breakdown

Configuration and templates

- `FitnessLevel` extends `LookupModel` (Id, Description). Adds Qualification, StrengthLoading, WeeklyTraining, MinSessionsPerWeek, MaxSessionsPerWeek, Reference. Seeded Novice, Intermediate, Advanced.
- `TrainingGoal` extends `LookupModel`. Seeded Fat loss, Strength, Size.
- `WorkoutPeriod` extends `LookupModel` and adds StartTime and EndTime. Seeded by the migrator, not by `HasData`.
- `TemplateWorkout` is a named plan at one fitness level, with notes, a reference URL, and exercises.
- `TemplateTrainingGoal` links a template to a goal. Composite key. Recommendation metadata, seeded by template name.
- `TemplateExercise` links a template to an `Exercise`. Unique on template plus exercise.
- `TemplateSet` is an ordered set on that exercise. Unique on exercise plus order.
- `TemplateSetMetric` stores a planned numeric or text value for a `Metric`.
- `Exercise` has a name, `ExerciseType`, steps, and a video link.
- `ExerciseMetric` says which metrics apply to an exercise. Fluent configuration uses the composite key ExerciseId plus MetricId.
- `Metric` has name, unit, and `MetricValueType`.

`WorkOut`, `WorkoutExercise`, and `WorkoutSet` are classes in the model project. They are not `DbSets` on `EntityDBContext`. The live performed workout is the `Actual*` family.

User planning

- `UserTrainingPreference` is one row per user. Unique `UserId`. Fitness level is required and restricted on delete.
- `UserTrainingPreferenceGoal` is the goal junction. Composite key. Cascade from the preference, restrict from the goal.
- `DOWWorkout` is one live planned session: user, `DayOfWeek`, period, template, sort order. It does not point at a proposal or an actual workout.

Schedule proposal

- `ScheduleProposal` stores user, `ProposalStatus` (Draft 1, Accepted 2, Abandoned 3), fitness level, recommended count, explanation, and timestamps. A filtered unique index allows one Draft per user.
- `ScheduleProposalGoal` snapshots the goals used for that proposal. Composite key, no separate Id.
- `ScheduleProposalSlot` is one recommended or edited session. Unique on proposal, day, period, and template. Day plus period is not unique.

Actual and historical

- `ActualWorkout` stores who performed it, the day, the workout name, an optional template id, start and end, notes, and `WorkOutStatus`.
- `ActualExercise` stores the exercise id plus `ExerciseName` and `ExerciseType` text.
- `ActualSet` stores execution order, completed flag, and an optional `TemplateSetId`.
- `ActualSetMetric` stores planned text and number beside actual text and number.

4. How it works in practice

A Novice user who wants Strength has one preference row and one preference-goal row. Generate creates a proposal row, proposal-goal rows, and slot rows that point at existing templates. Accept deletes that user's `DOWWorkout` rows and inserts new ones from the slots. When the user later logs Monday's session, a new `ActualWorkout` stores the template's name and an optional template id. Completing a set adds exercise, set, and metric rows. Changing the template afterwards changes what a future log would copy. It does not update the actual rows already stored.

6. Template, schedule, and actual architecture

1. High-level overview

These are three different records of a workout.

- **Template** answers what could be done: a named workout, its level, its exercises, and the planned metric values.
- **Schedule** answers what is planned for this user: a day, a period, and a pointer to a template. A proposal is a candidate schedule. `DOWWorkout` is the live schedule.
- **Actual** answers what happened: a started session, the name copied at the start, and the planned and actual metric values copied when a set was completed.

```
TemplateWorkout
   ↓  chosen by id
Schedule proposal slot, then DOWWorkout
   ↓  name, day, and template id copied at start
ActualWorkout
   ↓  exercise name and type copied when the set is completed
ActualExercise
   ↓
ActualSet  (optional link back to TemplateSet)
   ↓
ActualSetMetric  (planned values and actual values)
```

Nothing in this chain is one object that is updated from template through to history. Accept replaces plan rows. Logging inserts history rows. Template edits stay on the template tables.

2. Architecture at a glance

Question	Where it lives	What is stored
What is in a Push workout?	Template*	Structure and planned targets
What should this user do on Monday morning?	DOWWorkout	User, day, period, template id, sort
What did the advisor suggest before the user agreed?	ScheduleProposal*	Draft, then Accepted or Abandoned
What did the user do today?	Actual*	Snapshot name, copied plans, entered actuals, status

3. Detailed breakdown

`DOWWorkout` fields are `UserId`, `DayOfWeek`, `WorkoutPeriodId`, `TemplateWorkoutId`, and `SortOrder`. The day is `System.DayOfWeek` stored as an int. Sunday is 0. There is no `ScheduleProposalId` and no `ActualWorkoutId`.

`ActualWorkout` copies, at `SaveNewActualWorkOut`, the schedule row's day, template id, and template name into `DayOfWeek`, `TemplateWorkoutId`, and `WorkoutName`. Start date, start time, notes, user, and status Started are set then. The template relationship is optional and `OnDelete(DeleteBehavior.SetNull)`. If the template is later deleted, the actual row can remain and `WorkoutName` still holds the name that was copied. The relationship to `WorkOutStatus` is Restrict.

`ActualExercise` copies `Exercise.Name` into `ExerciseName` and the type description into `ExerciseType`, and keeps `ExerciseId`. That foreign key is Restrict, so an exercise that has been logged cannot be deleted while the actual row points at it. The parent workout cascade-deletes its actual exercises.

`ActualSet` keeps `ExecutionOrder`, `Completed`, and optional `TemplateSetId`. Deleting the template set sets that foreign key to null. The set row remains. `ActualSetMetric` writes `PlannedTextValue`, `PlannedNumericValue`, `ActualTextValue`, and `ActualNumericValue`. Planned numbers come from the template metric at completion time. Later edits to `TemplateSetMetric` do not flow into these columns.

`ResetActualWorkOut` marks the current actual workout Abandoned and deletes its exercises, sets, and metrics with `ExecuteDeleteAsync`. That is a deliberate wipe of an in-progress log, not a rewrite of history from the template.

4. How it works in practice

```
Template "Beginner Full Body" has squats, 3 sets, 8 reps
  ↓
DOWWorkout says Monday, Morning, that template id
  ↓
User starts Today's Workout
  ↓
ActualWorkout.WorkoutName = "Beginner Full Body"
```

```
ActualWorkout.TemplateWorkoutId = that id
ActualWorkout.DayOfWeek = Monday
  ↓
User completes set 1 and types the weight they used
  ↓
ActualExercise.ExerciseName = the exercise name at that moment
ActualSet.TemplateSetId = the template set id
ActualSetMetric.PlannedNumericValue = 8
ActualSetMetric.ActualNumericValue = the entered load or reps
  ↓
Someone later renames the template
  ↓
Workout records still show "Beginner Full Body" and the planned 8
```

7. Personalized schedule feature

1. High-level overview

The feature produces a weekly plan the user can refuse. Preferences say who the plan is for in training terms: a fitness level and any combination of Fat loss, Strength, and Size. Generate asks a catalogue advisor for sessions. The result is a draft proposal, not the live week. The user can change days, periods, templates, and how many sessions there are. Save stores that draft. Accept is the only step that replaces `DOWWorkout`. Abandon marks the draft abandoned and leaves the live week alone.

```
User enters preferences
        ↓
Generate recommendation
        ↓
Review proposal
        ↓
Modify proposal in memory
        ↓
Save proposal          or Accept, which saves and then replaces the live week
        ↓
DOWWorkout rows for that user only
```

The proposal exists so a recommendation and the user's edits have somewhere to live before the plan that Today's Workout reads is changed. Accepted and Abandoned proposals remain as history of that decision. One user has at most one Draft, enforced by a filtered unique index.

2. Architecture at a glance

```
GenerateSchedule.razor
  ↓
GenerateScheduleWorkflow
  ↓
TrainingPreferenceService          ScheduleProposalService
                                      ↓                ↓
                                CatalogScheduleAdvisor   ScheduleProposalValidator
                                      ↓
                                ScheduleProposal + goals + slots
                                      ↓  only on Accept
                                DOWWorkout
```

Four different judgements:

Kind	What it does	Can it block?
Recommendation	Advisor picks a count, days, Morning, and templates from the filtered pool	It only proposes
User choice	Any existing template, any period including Evening, any day including Sunday, any session count	The user is in control
Technical validation	Known template, known period, real day, at least one slot, no exact duplicate	Yes. Invalid input is not stored and is not accepted
Live schedule	DOWWorkout	Written only by Accept, or by the separate Workout Schedule page

3. Detailed breakdown

Types involved are `TrainingPreferenceService`, `IScheduleAdvisor`, `CatalogScheduleAdvisor`, `ScheduleProposalService`, `ScheduleProposalValidator`, `ScheduleProposal`, `ScheduleProposalGoal`, `ScheduleProposalSlot`, `ScheduleProposalVM`, and `ScheduleProposalSlotVM`. The page does not edit `ScheduleProposalSlot` entities. It edits the view model. The workflow maps that list to `ProposalSlotInput`.

Generate. If a draft exists, the page asks for confirmation using `ReplaceDraftConfirmation`. On confirm, preferences are upserted, then `GenerateAsync` runs. The advisor sees only the recommendation pool. The validator still checks ids against the full catalogue so a later edit is not limited to that pool. The new draft is inserted only after validation. Replacement deletes the old draft's slots and goals, saves that delete, then inserts the new draft, all inside one transaction. The delete is saved before the insert because SQL Server's filtered unique index cannot hold two Draft rows for one user.

Draft and edit. `GetDraftAsync` returns the proposal with fitness level, goals, periods, and template levels. The page holds `ScheduleProposalVM`. Dialog Submit updates that object. A new slot's `DayOfWeek` default is Sunday, which is valid. The page sets sort order

to the count plus one and does not invent a period or template, so an unfinished row fails validation.

Save. `UpdateSlotsAsync` replaces slots on the draft. Recommended session count stays as originally generated. Session count may differ from it. That difference becomes a warning.

Accept. The page passes the current slot list. The workflow saves it. If save fails, accept is not called, the live week is unchanged, and the stored draft is unchanged. If save succeeds, `AcceptAsync` re-reads the stored draft, checks the user, checks Draft, removes that user's `DOWWorkout` rows with `StageRemove`, adds new rows with the authenticated user id and the slot's day, period, template, and sort, marks the proposal Accepted, and saves once. Actual rows are not touched. The page then navigates to `/manage/workoutschedule`.

Abandon. Status becomes Abandoned. Slots on the screen are not written. The live week is not written.

Ownership. Every query that mutates a draft filters by the user id argument. Accepting another user's proposal id returns "There is no draft proposal to accept." A proposal that is no longer Draft cannot be accepted. Inside the transaction the code checks again and throws `InvalidOperationException` if the tracked row is no longer a draft. New proposal rows set `FitnessLevelId` only, so EF does not insert a duplicate fitness level. New `DOWWorkout` rows are created without attaching untracked navigations.

Catalog advisor, again in this feature's terms. Count is the lower middle of the level's min and max. Period is Morning. Days are spread across the week rather than packed from Monday. Templates in the request are ranked, then cycled. The service has already limited that request to the user's level and overlapping goals. Warnings after the fact tell the user when they leave that recommendation. They do not undo the edit.

4. How it works in practice

```
User deletes Wednesday in the editor and presses Accept without Save changes
  ↓
GenerateSchedule.razor passes _state.Proposal.Slots
  ↓
GenerateScheduleWorkflow.AcceptAsync calls SaveSlotsAsync
  ↓
UpdateSlotsAsync validates and replaces TRN_ScheduleProposalSlot
  ↓
ScheduleProposalService.AcceptAsync loads those saved slots
  ↓
StageRemove this user's TRN_DOWWorkout rows
  ↓
StageAdd the remaining days only
  ↓
Status = Accepted, one SaveChanges, commit
  ↓
Today's Workout will see the new week. Past ActualWorkout rows stay as they were
```

8. Workout execution and logging flow

1. High-level overview

Today's Workout turns the live plan into a log. It looks at the clock and the user's `DOWWorkout` rows, shows the template's exercises and planned metrics, and writes `Actual*` rows as the user starts the session and completes sets. The screen is a reflected grid of logging view models, with a child row per set, not `ManageGrid`.

```
DOWWorkout for this user, this day, this period
       ↓
User starts the workout
       ↓
ActualWorkout  (name and template id copied)
       ↓
User completes a set
       ↓
ActualExercise  (name and type copied)
       ↓
ActualSet
       ↓
ActualSetMetric  (planned and actual)
```

2. Architecture at a glance

```
MyNavMenu "Today's Workout"
  ↓
Manage.razor entity logworkout
  ↓
LogWorkout.razor
  ↓
ILogWorkOutService / LogWorkOutService
  ↓
new EntityDBContext per operation, plus queries on the service context
  ↓
TRN_DOWWorkout, template tables, TRN_Actual*
```

3. Detailed breakdown

`LoadDOWDataPerUserCurrentDateTime` loads all `DOWWorkout` rows through `GetAllAsync` and then filters in memory. A row matches when the user id matches, the day is `DateTime.Now.DayOfWeek`, and the current time of day is greater than the period start and less than or equal to the period end. If several rows match, the caller's list contains all of them; the first match is what the screen treats as the session when it takes one. Evening is a known mismatch: the period comparison used here does not treat an end time that is stored before the start as an overnight window, so an Evening row is not selected by this query. The generator warns about that and does not change the period table.

`GetCurrentWorkout` finds an actual workout for this user, today, whose start time falls inside the current period. `SaveNewActualWorkOut` returns that id when one exists. Otherwise it reads the chosen `DOWWorkout`, copies day, template id, and template name, sets status Started, and `InsertAsync`s the actual workout.

`GetExercises` and `GetExerciseData` build `LogWorkoutLogExerciseVM` and `LogWorkoutSetVM` from the template. Planned metric text is assembled for display. If an actual metric already exists for that set, its actual values are copied back onto the view model so a resumed session shows what was entered.

`CompleteSet` reads the template exercise's current name and type. It inserts or updates `ActualExercise` for that exercise on this actual workout, then inserts or updates `ActualSet` with the template set id, execution order, and completed flag. When the set is marked complete, each `LogWorkoutSetMetricVM` is written to `ActualSetMetric` with both planned and actual values. Each of those writes is its own `InsertAsync` or `UpdateAsync` on its own context. There is no transaction around the exercise, the set, and the metrics.

`CompleteWorkout` sets status Ended and writes the end date and time. `ResetActualWorkOut` sets Abandoned and deletes the child actual rows.

History reads `ActualWorkout` through `WorkoutHistory` and shows `WorkoputHeaderDisplayVM`. It displays the stored name, day, start, end, status, and notes. It does not rejoin the live template to replace the snapshot name.

4. How it works in practice

```
Monday morning, a DOWWorkout points at Beginner Full Body
  ↓
LogWorkout calls LoadDOWDataPerUserCurrentDateTime
  ↓
The Morning period contains DateTime.Now, so the row is shown
  ↓
Start calls SaveNewActualWorkOut
  ↓
TRN_ActualWorkout gains a Started row named Beginner Full Body
  ↓
The grid lists template exercises and sets
  ↓
Complete on a set calls CompleteSet
  ↓
TRN_ActualExercise, TRN_ActualSet, and TRN_ActualSetMetric are inserted
  ↓
End calls CompleteWorkout
  ↓
Workout Records lists that header from the actual row
```

9. Database and EF Core architecture

1. High-level overview

Entity classes in `GymTrackerDataModel` become tables through `EntityDBContext`. Fluent configuration in `OnModelCreating` sets table names, keys, indexes, delete behaviour, precision, and the two `HasData` catalogues. EF Core migrations in the data-model project create and alter the SQL Server schema. Identity uses a second context and its own migration in the web project. Both contexts use `DefaultConnection`.

```
Entity classes
    ↓
EntityDBContext  (and ApplicationDbContext for Identity)
    ↓
OnModelCreating and migrations
    ↓
SQL Server
```

2. Architecture at a glance

DbSet	Table	Entity
LKP_Exercise	LKP_Exercise	Exercise
LKP_ExerciseType	LKP_ExerciseType	ExerciseType
LKP_TemplateExercise	LKP_TemplateExercise	TemplateExercise
LKP_TemplateSet	LKP_TemplateSet	TemplateSet
LKP_WorkoutTemplate	LKP_WorkoutTemplate	TemplateWorkout
LKP_TemplateSetMetric	LKP_TemplateSetMetric	TemplateSetMetric
LKP_Metric	LKP_Metric	Metric
LKP_ExerciseMetric	LKP_ExerciseMetric	ExerciseMetric
LKP_WorkoutPeriod	LKP_WorkoutPeriod	WorkoutPeriod
LKP_WorkOutStatus	LKP_WorkOutStatus	WorkOutStatus
LKP_FitnessLevel	LKP_FitnessLevel	FitnessLevel
LKP_TrainingGoal	LKP_TrainingGoal	TrainingGoal
LKP_TemplateTrainingGoal	LKP_TemplateTrainingGoal	TemplateTrainingGoal
USR_TrainingPreference	USR_TrainingPreference	UserTrainingPreference
USR_TrainingPreferenceGoal	USR_TrainingPreferenceGoal	UserTrainingPreferenceGoal
TRN_DOWWorkout	TRN_DOWWorkout	DOWWorkout
TRN_ScheduleProposal	TRN_ScheduleProposal	ScheduleProposal
TRN_ScheduleProposalGoal	TRN_ScheduleProposalGoal	ScheduleProposalGoal
TRN_ScheduleProposalSlot	TRN_ScheduleProposalSlot	ScheduleProposalSlot
TRN_ActualWorkout	TRN_ActualWorkout	ActualWorkout
TRN_ActualExercise	TRN_ActualExercise	ActualExercise
TRN_ActualSet	TRN_ActualSet	ActualSet
TRN_ActualSetMetric	TRN_ActualSetMetric	ActualSetMetric

3. Detailed breakdown

Composite keys. `ExerciseMetric` (ExerciseId, MetricId). `UserTrainingPreferenceGoal`, `TemplateTrainingGoal`, and `ScheduleProposalGoal` are junctions of the two parent ids. `ScheduleProposalGoal` has no Id column.

Unique indexes. Preference `UserId`. Template exercise on template plus exercise. Template set on template-exercise plus order. Proposal slot on proposal, day, period, and template. Proposal `UserId` filtered to `[Status] = 1`, so only one Draft. There is no unique index on day plus period alone, and none on user plus day plus period for `DOWWorkout`.

Cascade. Exercise to exercise type. Template composition from template to exercise to set to set metric. Exercise metrics when the exercise goes. Preference goals when the preference goes. Template goals when the template goes. Proposal goals and slots when the

proposal goes. Actual exercise, set, and set metric down from the actual workout.

Restrict. Template set metric to metric. Template to fitness level. Preference to fitness level. Preference goal to training goal. Template goal to training goal. Proposal to fitness level. Proposal goal to training goal. Proposal slot to period and to template. Actual workout to status. Actual exercise to exercise. Actual set metric to metric.

SetNull. `ActualWorkout.TemplateWorkoutId` when the template is deleted. `ActualSet.TemplateSetId` when the template set is deleted. History can outlive the template row. The copied name and planned numbers remain on the actual tables.

DOWWorkout foreign keys are not given an explicit `OnDelete` in `OnModelCreating`. The class has required `WorkoutPeriodId` and `TemplateWorkoutId`, so EF convention creates those relationships. There is no foreign key to `AspNetUsers`, to `ScheduleProposal`, or to `ActualWorkout`.

Precision. `TemplateSetMetric.NumericValue` is decimal(10, 2).

Seed. Fitness levels and training goals are `HasData` in the context. The migrator's `SeedTemplateTrainingGoals` links templates to goals by name after workouts are rebuilt. `SeedNoviceSchedule` deletes `TRN_DOWWorkout` rows and inserts a novice week. Lookup periods, metrics, exercises, and templates are also seeded from that console program.

Migrations in the data model. `20260930114912_initialClean`, `20260930121605_LinkActualWorkoutToTemplate`, `20261001060115_AddTemplateWorkoutReference`, `20261001070608_AddFitnessLevel`, `20261002054845_AddScheduleGeneratorFoundation`. Identity schema lives under `GymTracker/Data/Migrations`.

Tracking and queries. `GetAllAsync` is untracked with one-level navigation includes. Scheduling reads that must be edited use tracked `QueryAsync`. Reads returned to the page use untracked queries. `OnConfiguring` applies SQL Server from `appsettings.json` only when the context was constructed without options. The web host and `Helpers.BuildOptions` always pass options, so that fallback is for the parameterless design-time path. `EntityDBContextFactory` supports EF tools.

4. How it works in practice

Deleting a template is restricted while a proposal slot or a live `DOWWorkout` still points at it, and it is restricted by the template's fitness-level relationship in the other direction: the level cannot be removed while templates use it. Deleting a template that only historical workouts reference nulls `ActualWorkout.TemplateWorkoutId` and leaves `WorkoutName`. Deleting an exercise that an `ActualExercise` still references is refused. Deleting a proposal removes its goals and slots and does not remove `DOWWorkout` or actual rows.

10. Transaction and consistency model

1. High-level overview

Consistency is explicit only on the scheduling workflows. Those services start a transaction on the shared context, stage every related change, save, and commit. A failure rolls the transaction back and rethrows. Everything else saves one row at a time. A multi-row template edit or a completed set can therefore stop halfway, with earlier inserts already committed.

On SQL Server the scheduling transaction is a real database transaction. In the in-memory test provider, begin, commit, and rollback do nothing, and consistency is "SaveChanges finished, or it threw before the store changed."

2. Architecture at a glance

```
Scheduling write
  Business service owns the transaction
    ↓
  BeginTransactionAsync on one IGenericRepoService
    ↓
  StageAdd / StageRemove on several repositories sharing EntityDBContext
    ↓
  SaveAsync  →  SaveChangesAsync
    ↓
  CommitTransactionAsync
    or RollbackTransactionAsync and throw

Older write
  InsertAsync / UpdateAsync / DeleteAsync
    ↓
  SaveChanges immediately, no transaction
```

3. Detailed breakdown

Operation	Owner	Repositories that participate	What one commit covers
Save preferences	TrainingPreferenceService.SaveChangesAtomicallyAsync	Begin/Save/Commit on the preference repository. Goal rows were staged on the preference-goal repository.	Preference insert or update and goal adds and removes
Replace draft	ScheduleProposalService.SaveAtomicallyAsync	Proposal, slot, and goal repositories	Delete of the old draft (after its children are staged removed) and insert of the new draft, goals, and slots. Two SaveAsync calls inside the one transaction
Update slots	ScheduleProposalService.SaveAtomicallyAsync	Slot repository, tracked proposal	Delete of old slots, then insert of the new slots. Two saves inside the transaction
Abandon	ScheduleProposalService.SaveAtomicallyAsync	Proposal repository	Status and AbandonedAt
Accept	ScheduleProposalService.SaveAtomicallyAsync	DOW repository for removes and adds; proposal repository for SaveAsync	Replacement of that user's live week and the proposal becoming Accepted
Accept from the page	GenerateScheduleWorkflow, then the service	The slot update transaction runs first. The accept transaction runs only after it succeeds.	Two transactions in sequence. If accept throws after the slot save, the draft has the new slots and the live week is unchanged
Template save, exercise save, manual schedule save, log start, complete set, complete workout	The feature service method	A new GenericRepoService and a new context per call	One row, or a sequence of one-row commits with no shared transaction

Accept's transaction re-checks that the tracked proposal still belongs to the user and is still Draft before it stages the live-week replacement. The workflow does not put the slot save and the accept into that same transaction.

Draft replacement relies on ordering inside the transaction: the old Draft is saved as deleted before the new Draft is inserted. Doing the insert first can violate `IX_TRN_ScheduleProposal_UserId`. The two saves stay in one transaction so a failed insert rolls the delete back.

User isolation is by the Guid passed from the page, not by a database foreign key. Accept and draft queries include that Guid. The live-week delete is `UserId == userId`. Another user's `DOWWorkout` rows are not staged.

4. How it works in practice

```
User accepts a draft that removed one day
  ↓
Transaction A: delete old slots, insert the remaining slots, commit
  ↓
Transaction B begins
  ↓
Tracked draft is still Draft for this user
  ↓
StageRemove every DOWWorkout for this user
  ↓
StageAdd one DOWWorkout per remaining slot
  ↓
Proposal.Status = Accepted, AcceptedAt = UtcNow
  ↓
SaveChanges
  ↓
Commit
  ↓
ActualWorkout, ActualExercise, ActualSet, and ActualSetMetric are outside both transactions
```

If validation fails before transaction A, neither transaction starts. The stored draft and the live week stay as they were. If transaction B fails, transaction A has already committed the edited draft, and the live week has not been replaced.

End of the architecture description. Source of truth: the GymTracker solution as read for this document.